Sprint 2 - Foundation & Design

Team 4 - Nishnath Bandari, Nisarg Jaswal, Ryan Tersigni, Mattrey Kumar Rana

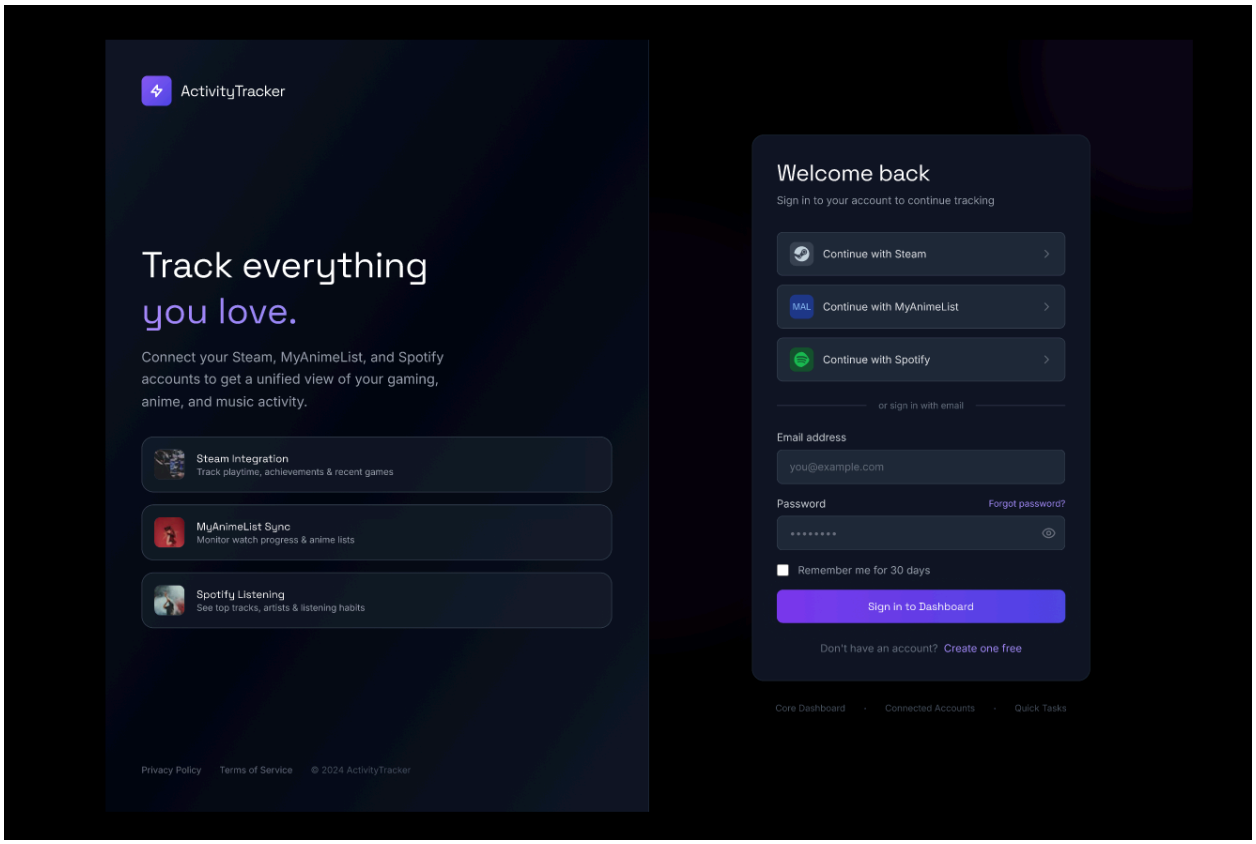
Index

---

1. UI Mockups & Core User Flow
2. High-Level Database Model
3. Key Epics & User Stories
4. Technical Risks & Dependencies

UI Mockups & Core User Flow

Link to Figma:
https://www.figma.com/proto/s3iaE7mzVpVQQmRkf6vsya/Activity-Tracker-App?node-id=0-1&t=Sse7l2DK3BkXRrcJ-1



UI mockup Scenario 1 : User Navigates Through the Core Dashboard Flow

Goal:

Show how a typical user interacts with the main features of the app.

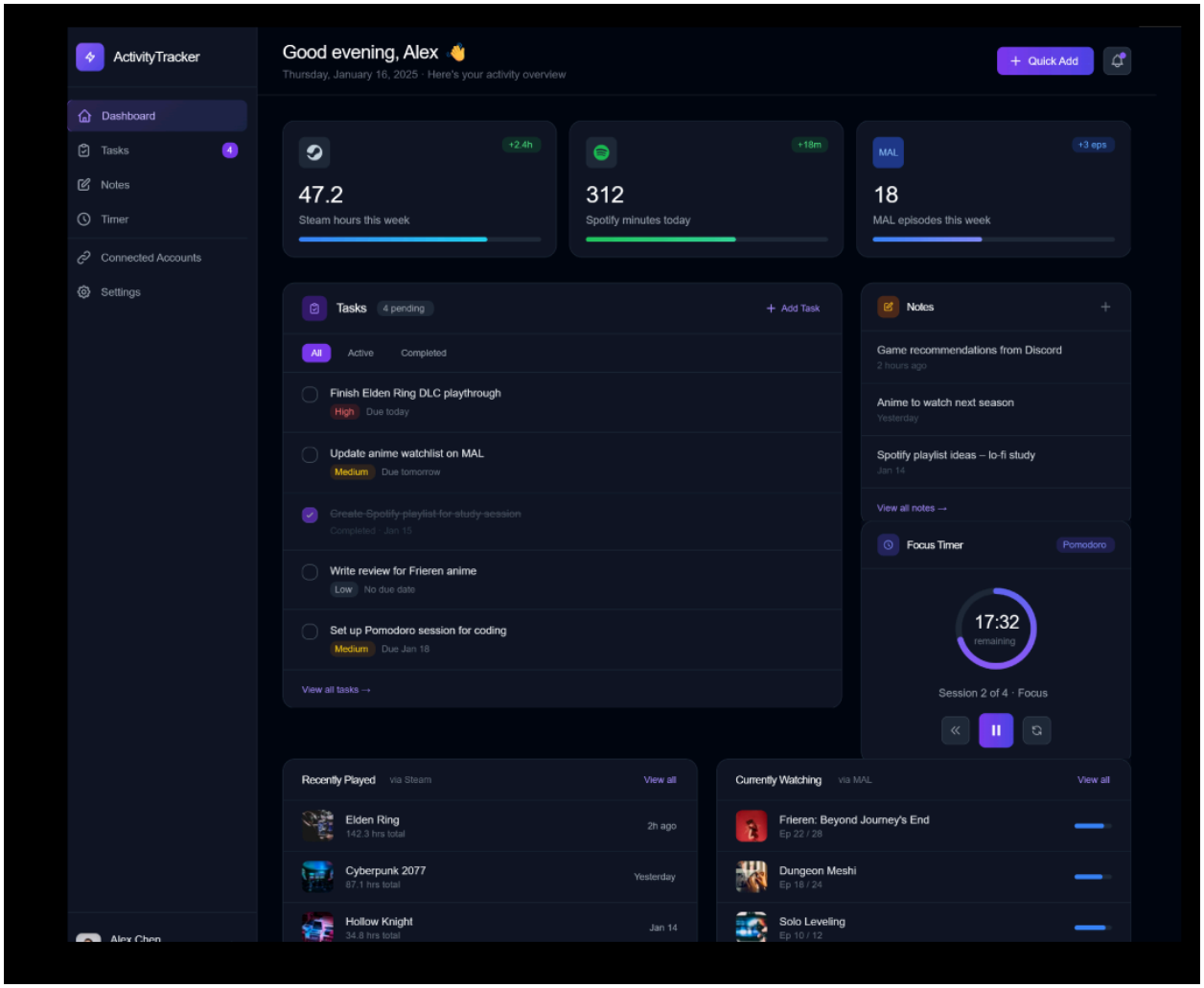
Steps in the Scenario:

1. User opens the app and lands on the login screen.
2. User signs in using email/password or an external account (Steam, MAL, Spotify).
3. After login, the user is taken to the main dashboard, where all widgets are displayed.

4. The user rearranges widgets (tasks, notes, timer, activity stats) based on personal preference.
5. The user clicks on the Task widget to add or update a task.
6. The user interacts with the Notes widget to jot down a quick reminder.
7. The user starts a focus session using the built-in timer.
8. The user checks activity summaries (e.g., weekly game time, anime progress, music stats).
9. If an external service fails to sync, the user receives a notification alert.
10. The user logs out or closes the app.

Expected Outcome:

A clear understanding of how users move through the app.



UI mockup Scenario 2 : Scenario: User Connects External Accounts and Views Synced Data

Goal:

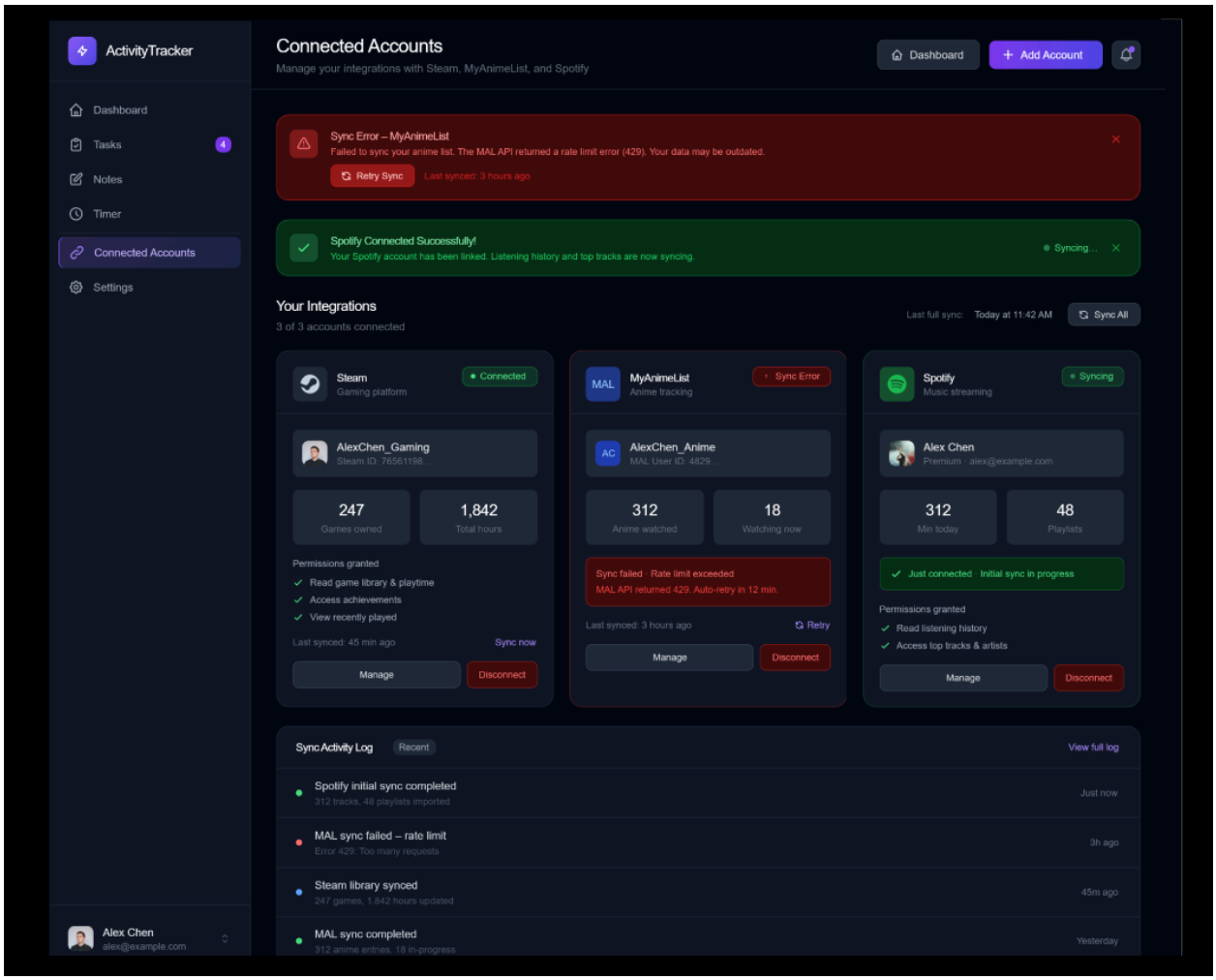
Show how a user links third-party services and how the UI responds when data syncs successfully or fails.

Steps in the Scenario:

1. User opens the “Connected Accounts” section from the dashboard.
2. User selects a service (Steam, MAL, Spotify) and clicks “Connect Account.”
3. A pop-up appears explaining what data will be accessed.
4. User completes the authentication process through the external service.
5. The UI updates to show a “Connected” status and last sync time.
6. The dashboard refreshes to display new activity data (games played, anime progress, music stats).
7. If a sync fails, the user sees a clear notification with a retry option.
8. User can disconnect the account at any time, and the UI updates immediately.

Expected Outcome:

A clear representation of how users connect external accounts and how synced data appears in the UI, including how errors are communicated visually.



UI mockup Scenario 3 : Scenario: User Creates Quick Tasks and Notes From the Dashboard

Goal:

Show how users perform fast actions with minimal steps, supporting usability and productivity expectations.

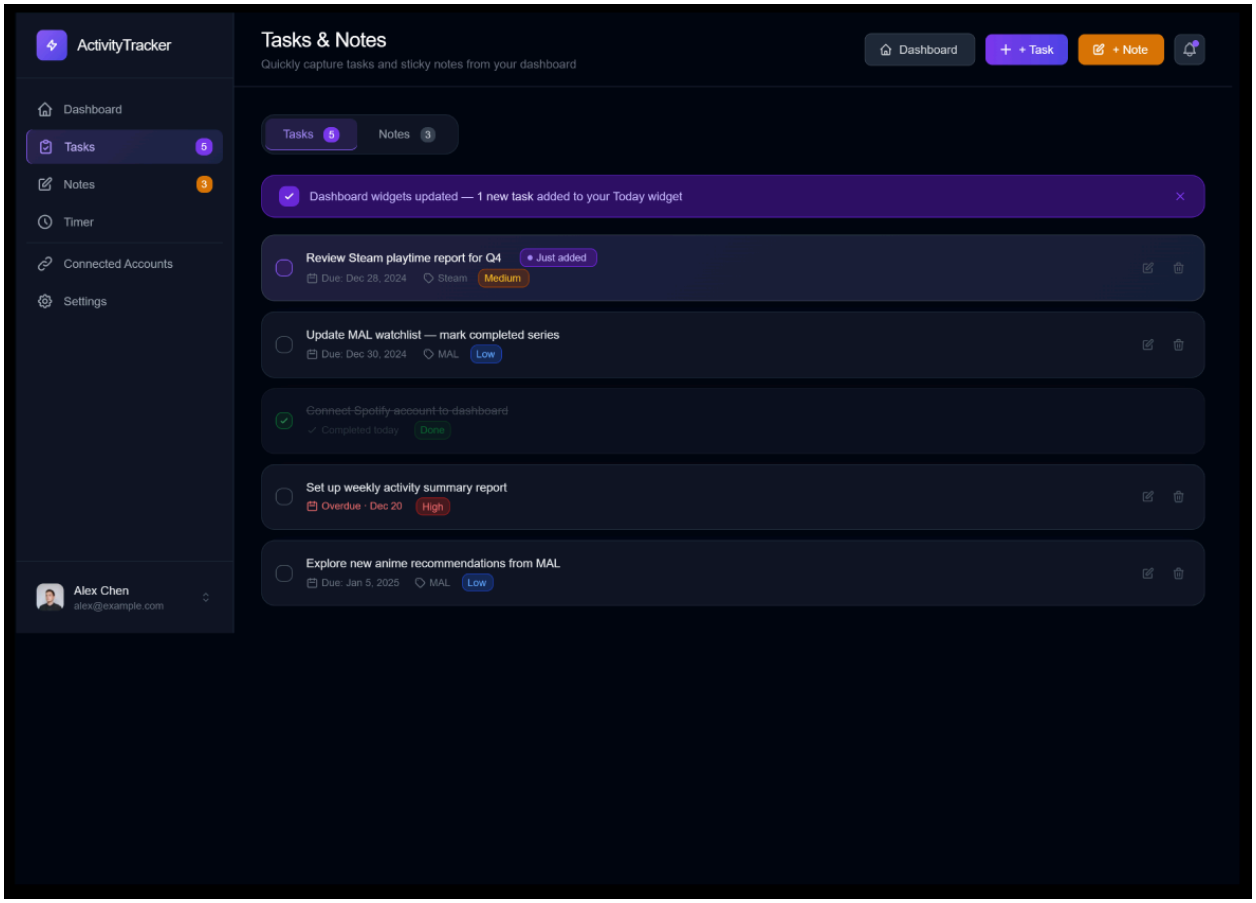
Steps in the Scenario:

1. User clicks the “+ Task” button on the dashboard.
2. A small input panel appears where the user enters a task name and due date.
3. User saves the task, and it appears instantly in the Task widget.
4. User clicks the “+ Note” button to create a quick sticky note.
5. A small pop-up appears where the user types a reminder and pins it.

6. Both actions provide immediate visual feedback (highlight, checkmark).
7. User can edit or delete tasks and notes directly from the widget.
8. All changes save automatically without refreshing the page.

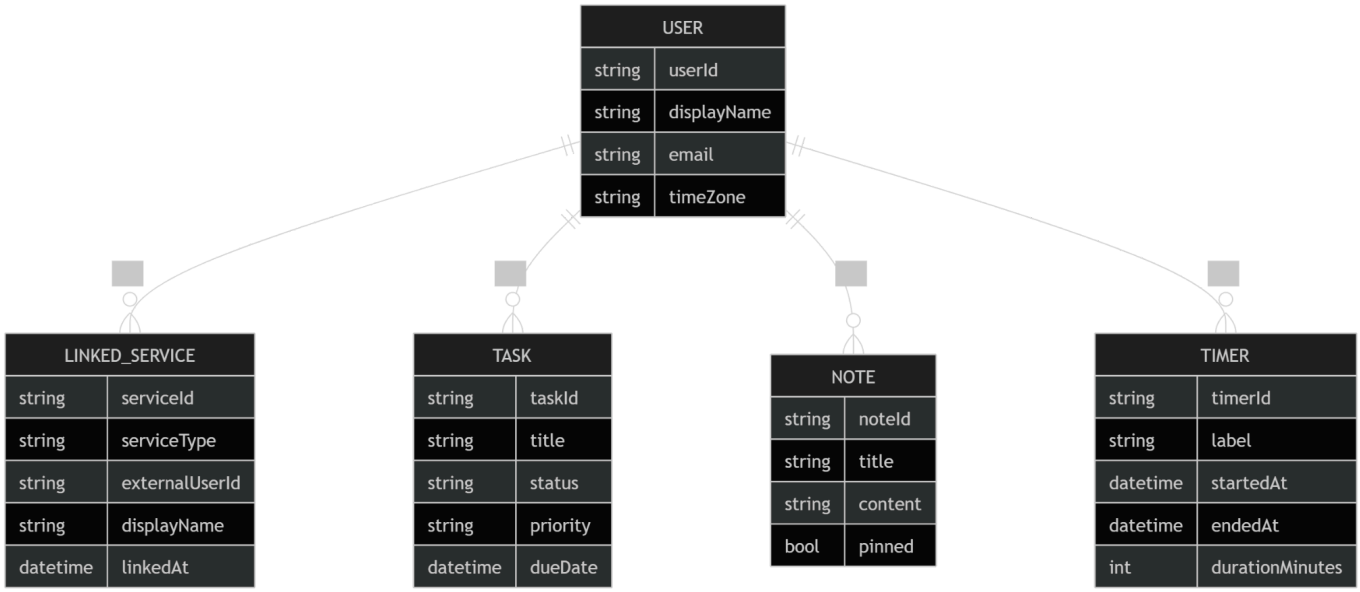
Expected Outcome:

A UI flow that demonstrates fast, simple interactions for creating tasks and notes, ensuring the mockups support usability and efficiency requirements.



High-Level Database Model

ERD (Entity Relationship Diagram)



1) Collections (and Subcollections)

Top-level collection

- `users`
    - One document per authenticated user (ID = Firebase Auth `uid`).

User-scoped subcollections (under `users/{userId}`)

- `linkedServices` – External accounts (Steam, MAL/Anilist, Spotify) linked by the user
- `tasks` – User's personal tasks / to-dos
- `notes` – Sticky notes for dashboard
- `timers` – Timer sessions (Pomodoro-style) and/or configurations

Effective "collection list":
`users`, `users/{userId}/linkedServices`, `users/{userId}/tasks`, `users/{userId}/notes`, `users/{userId}/timers`

---

2) Document Structure (Sample JSON)

a) `users/{userId}`

```
{
 "displayName": "Ryan",
 "email": "ryan@example.com",
 "createdAt": "2026-02-28T15:30:00Z",
 "avatarUrl": null,
 "themePreference": "dark",
 "timeZone": "America/Toronto"
}
```

b) `users/{userId}/linkedServices/{serviceId}`

```
{
 "serviceType": "steam",
 "displayName": "ryan_plays",
 "externalUserId": "76561198000000000",
 "linkedAt": "2026-03-01T18:00:00Z",
 "scopes": ["read_profile", "read_game_stats"],
 "lastSyncedAt": "2026-03-02T10:15:00Z",
 "isActive": true
}
```

c) `users/{userId}/tasks/{taskId}`

```
{
 "title": "Finish Sprint 2 data model",
 "description": "Complete NoSQL output document and push to GitHub.",
 "status": "in_progress",
 "priority": "high",
 "dueDate": "2026-03-02T23:59:00Z",
 "createdAt": "2026-03-01T13:00:00Z",
 "updatedAt": "2026-03-02T09:45:00Z",
 "tags": ["school", "UAT"]
}
```

d) `users/{userId}/notes/{noteId}`

```
{
 "title": "Dashboard layout ideas",
 "content": "Left: tasks & notes. Right: Steam and Spotify widgets.",
 "pinned": true,
 "createdAt": "2026-03-01T14:20:00Z",
 "updatedAt": "2026-03-01T14:30:00Z",
 "color": "#FFD966"
}
```

e) `users/{userId}/timers/{timerId}`

```
{
 "label": "Study Session",
 "startedAt": "2026-03-02T16:00:00Z",
 "endedAt": "2026-03-02T16:25:00Z",
 "durationMinutes": 25,
 "status": "completed",
 "linkedTaskId": "task123"
}
```

---

3) Relationship Strategy (Embedding vs Referencing)

Strategy:

- **User as root entity**, everything else as **subcollections** under the user

- Avoid large embedded arrays; use separate documents for growing lists (tasks, notes, timers, linked services)

Entity	Stored As	Relationship	Why
User	`users/{userId}`	Root	Identity + profile
Linked Service	`users/{userId}/linkedServices/{serviceId}`	1 user → many services	Small list per user, easy to query & secure
Task	`users/{userId}/tasks/{taskId}`	1 user → many tasks	Potentially large, needs filtering & sorting
Note	`users/{userId}/notes/{noteId}`	1 user → many notes	Potentially many, dashboard operations
Timer Session	`users/{userId}/timers/{timerId}`	1 user → many sessions	High write volume over time

Why not embed everything in `users`?

- Tasks, notes, and timers can grow large and change frequently
- Big embedded arrays are bad for Firestore (doc size limits + heavy writes)
- Subcollections give:
    - Better query support (filter by status, dueDate, etc.)
    - Better scaling
    - Cleaner firestore security rules (`request.auth.uid == userId`)

So:

- **Embedded:** small, stable user profile fields (theme, time zone, etc.)
- **Referenced (via subcollections):** tasks, notes, timers, linked services

---

4) Access Patterns (Top Queries & Support)

Below are the **main queries** the app will use and how the model supports them.

AP1: Get current user's tasks (filtered and sorted)

Use case: Show the user's to-dos on the dashboard.

- **Query:**
  `users/{userId}/tasks`
  - Filter: `status == "todo"` (or `"in_progress"`)
  - Order by: `dueDate` ascending

Model support:
Tasks are stored as a subcollection per user, so no cross-user filtering needed. Firestore can efficiently query within `users/{userId}/tasks` by fields like `status` and `dueDate`.

AP2: Get current user's pinned notes

Use case: Show important sticky notes on the dashboard.

- **Query:**
  `users/{userId}/notes`
  - Filter: `pinned == true`
  - Order by: `updatedAt` descending

Model support:
Notes are per-user; pinned notes can be fetched cheaply with a simple filter and sort.

AP3: Get user's recent timer sessions

Use case: Show recent productivity sessions or build simple analytics (e.g., minutes focused today).

- **Query:**
  `users/{userId}/timers`
  - Filter: `status == "completed"`
  - Order by: `startedAt` descending
  - Limit: e.g., 20

Model support:
Timers are in a user-scoped subcollection, enabling quick retrieval of recent completed sessions.

AP4: List linked external services for a user

Use case: Display which platforms (Steam/MAL/Spotify) are connected and allow unlinking.

- **Query:**
  `users/{userId}/linkedServices`

Optional filter:

- Filter: `isActive == true`

Model support:
Linked services subcollection under each user makes it trivial to list a user's connections. No joins needed.

AP5: Dashboard load – combined fetch

Use case: Load dashboard with tasks, notes, linked services, and possibly timers.

- Parallel queries to:
    - `users/{userId}`
    - `users/{userId}/tasks` (filtered by status, limited)
    - `users/{userId}/notes` (pinned + recent)
    - `users/{userId}/linkedServices`
    - Optionally `users/{userId}/timers` (recent)

Model support:
All user data is scoped under `users/{userId}`, so the frontend can run multiple small queries in parallel and populate the dashboard quickly.

---

5) Index Plan (High Level)

Firestore automatically indexes single fields, but some **composite indexes** may be needed for combined filters + sorting.

Planned indexes (conceptual, not exact console syntax):

1. **Tasks – filter by status, order by dueDate**
    - Collection: `users/{userId}/tasks`

    - Composite index:
        - `status` (asc)
        - `dueDate` (asc)
2. **Supports:** “Get user’s upcoming TODO tasks ordered by due date”.

---

2. **Notes – filter by pinned, order by updatedAt**
    - Collection: `users/{userId}/notes`
    - Composite index:
        - `pinned` (asc or desc)
        - `updatedAt` (desc)
3. **Supports:** “Get pinned notes; show most recent first”.

---

3. **Timers – filter by status, order by startedAt**
    - Collection: `users/{userId}/timers`
    - Composite index:
        - `status` (asc)
        - `startedAt` (desc)
4. **Supports:** “Get recent completed sessions for analytics/history”.

---

4. **Optional: Tasks – tags-based filtering (if needed later)**
    - Collection: `users/{userId}/tasks`
    - Field index: `tags` (array-contains)
5. **Supports:** “Get tasks with a specific tag (e.g., ‘school’ or ‘gaming’)”.

Key Epics and User Stories

EPIC 1: Account Access & Authentication

Epic Description

Implement secure account creation, login, logout, and session persistence to protect user data and enable access to personalized dashboard features.

Business Value

- Enables secure personalization
- Protects user data
- Foundation for all protected features

US-01: Create Account

As a new user, I want to create an account so that I can access my personal dashboard.

Acceptance Criteria

- Registration page is accessible from landing page.
- Required fields (email, password) must not be empty.
- Email format validation is enforced.
- Password meets minimum security rules (e.g., length requirement).
- Successful registration redirects user appropriately.
- Failed registration displays a clear error message.
- Form data remains populated if validation fails.

Testing

- Test valid registration flow.
- Test invalid email format.
- Test weak password.
- Test duplicate email handling.
- Verify redirect behavior.
- Verify error message clarity.

US-02: Login and Logout

As a user, I want to log in and log out so that my data remains secure.

Acceptance Criteria

- Login page validates credentials.
- Invalid credentials show error message.
- Logout option is visible on dashboard.
- Logout clears session/token.
- Protected routes are inaccessible after logout.

Testing

- Test login with valid credentials.
- Test login with invalid credentials.
- Test logout functionality.
- Attempt direct URL access after logout.
- Verify session token removal.

US-03: Persistent Session

As a user, I want my session to remain active so that I do not need to log in repeatedly.

Acceptance Criteria

- Session persists after page refresh.
- Session expires securely after defined duration.
- Expired session redirects to login.
- Unauthorized users cannot access protected routes.

Testing

- Refresh page after login.
- Simulate session expiration.
- Attempt dashboard access when unauthenticated.
- Verify redirect timing (≤ 2 seconds).

EPIC 2: External Service Integration

Epic Description

Allow users to connect external accounts and sync activity data into a unified dashboard.

Business Value

- Reduces need for multiple platforms
- Increases engagement
- Differentiates product

US-04: Connect External Accounts

As a user, I want to connect my external accounts so that my activity data appears in one place.

Acceptance Criteria

- “Connect Accounts” section is accessible.
- User can connect at least one supported service (or mock API).
- UI displays connection status.
- User can disconnect an account.
- Privacy notice explains accessed data.

Testing

- Connect supported service.
- Verify connection status display.
- Disconnect service.
- Confirm data stops updating after disconnect.
- Verify privacy notice visibility.

US-05: Automatic Data Sync

As a user, I want my activity data to sync automatically so that my dashboard stays updated.

Acceptance Criteria

- Sync triggers on login or refresh.
- Loading indicator displays during sync.
- UI remains responsive during sync.
- Cached data loads if API fails.
- API rate limits are respected.

Testing

- Verify auto-sync on login.
- Verify loading state.
- Simulate API delay.
- Simulate API failure.
- Confirm cached fallback behavior.

US-06: Sync Failure Notification

As a user, I want to receive a notification if syncing fails so that I can reconnect my account.

Acceptance Criteria

- Error notification displays on sync failure.
- Retry/Reconnect option is available.
- App does not crash.
- Sensitive information is not exposed in errors.
- Notification clears after successful sync.

Testing

- Simulate API failure.
- Verify error message content.
- Test retry functionality.
- Inspect console/logs for token leakage.

- Verify recovery after successful retry.

EPIC 3: Unified Dashboard Experience

Epic Description

Provide a customizable dashboard where users can view and organize widgets for productivity and activity tracking.

Business Value

- Core product value delivery
- Improves user productivity
- Enhances retention through customization

US-07: Unified Dashboard View

As a user, I want a unified dashboard so that I can view my information in one place.

Acceptance Criteria

- Dashboard displays tasks, notes, timer, and activity widgets.
- Empty states are shown when no data exists.
- Dashboard loads within 3 seconds (typical conditions).
- Responsive layout (desktop + mobile).
- No horizontal scroll on 375–414px width.

Testing

- Verify widget rendering.
- Test empty state scenarios.
- Measure load performance.
- Resize to mobile viewport.
- Test at 200% zoom.

US-08: Customize Widget Positions

As a user, I want to rearrange widgets so that I can organize my dashboard based on my preferences.

Acceptance Criteria

- Widgets can be reordered (drag/drop or move controls).
- UI feedback appears during reordering.
- Layout remains functional after changes.
- Keyboard navigation is not broken.

Testing

- Move widgets.
- Refresh page and confirm order persists.
- Verify keyboard navigation.
- Confirm layout stability.

US-09: Persist Dashboard Layout

As a user, I want my dashboard layout to be saved so that I do not need to rearrange it every time.

Acceptance Criteria

- Layout saves automatically after change.
- Layout persists after refresh/login.
- Default layout loads if saved layout fails.
- Duplicate widgets do not appear.

Testing

- Reorder widgets and refresh.
- Logout/login and verify layout.
- Corrupt saved layout and verify fallback.
- Confirm no duplicate rendering.

Technical Risks & Dependencies

Risks:

Potential Backend Issues:

1. ERD or NoSQL schema might need some rework once actual features are implemented.
2. Potential scalability issues with current DB design, might not be able to handle larger user loads.
3. API endpoint mismatch, could slow frontend-backend integration (focus on this)
4. Authentication issues where session handling or token expiration could cause login/logout issues.
5. Missing constraints or poor relationship mapping could lead to duplicate or orphaned data.

Potential Frontend Issues:

1. Inconsistent design: UI mockups might not translate perfectly to implemented components.
2. Responsiveness problems: The layout may not display correctly on mobile or smaller screens.
3. Browser compatibility: Certain features may behave differently based on browser (Chrome, firefox, etc.)

Integration & Dependency Risks:

1. API integration failures: Using external APIs, downtime or rate limits might disrupt functionality if not implemented properly.
2. Environment setup issues: Local and production environments might behave differently (missing .env files etc.)

Deployment Risks:

1. Hosting Limitations: Free hosting sites may restrict DB size (for the sake of this project, try limiting data).
2. Security misconfigurations: Missing HTTPS or exposed API keys could create vulnerabilities.
3. Performance bottlenecking: Slow server response or large assets could affect load time.

General Risks:

1. Team coordination issues: Miscommunication b/w frontend and backend devs.
2. Incomplete documentation: Missing user stories, acceptance criteria, or test cases.
3. Feature powercreep: Adding way too many features during development
4. Testing gaps: Limited testing time might let bugs slip through.
5. Project overseer overseeing things by mistake: Everyone should keep an eye on deliverables and overall course of the project.

Dependencies:

1. Third-party APIs & Integrations - Steam, MAL/Anilist, Spotify
2. Authentication & Hosting - Firebase Auth, Firestore Database, and hosting sites (Vercel)
3. Environment Config - .env files, API keys, and service credentials required for local and production environments
4. UI Libraries/ Frameworks - React Native, React with dependency on component libraries.
5. Version Control CI/CD - GitHub for builds and deployment workflows

---